

# **CLASSI E OGGETTI**

Giovedì 5 Marzo 2026

# CLASSI

- Il concetto di classe è fondamentale per Java, poiché fornisce un modo per definire nuovi tipi definiti dall'utente, completi di funzioni e operatori. E' possibile separare le proprietà che sono essenziali per l'uso corretto di una classe dai dettagli necessari per la sua implementazione.
- Le classi sono fondamentali per la programmazione che utilizza un approccio orientato agli oggetti, che è uno specifico paradigma di programmazione

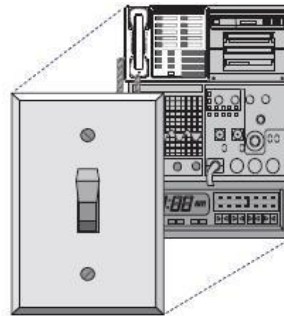


# PROGRAMMAZIONE AD OGGETTI

- Tra i principi dell'approccio orientato agli oggetti, che hanno lo scopo di facilitare la robustezza, l'adattabilità e la riusabilità del codice:
  - Astrazione
  - Incapsulamento
  - Modularità



Abstraction



Encapsulation



Modularity



# ASTRAZIONE

- La nozione di astrazione consiste nel ridurre un sistema complicato alle sue parti fondamentali e descrivere queste parti in un linguaggio semplice e preciso. Tipicamente, descrivere le parti di un sistema implica nominarle e spiegare la loro funzione
- Applicare l'astrazione alla progettazione di strutture di dati da luogo alla definizione delle strutture dati astratte (ADT).



# INCAPSULAMENTO

- Un altro importante principio della progettazione orientata agli oggetti è il concetto di incapsulamento, che afferma che i diversi componenti di un sistema software non dovrebbero rivelare i dettagli interni delle loro rispettive implementazioni.
- Uno dei principali vantaggi dell'incapsulamento è che dà al programmatore la libertà di implementare i dettagli di un sistema.
- L'unico vincolo per il programmatore è quello di mantenere l'interfaccia astratta che gli esterni vedono
- I dati dell'oggetto normalmente sono dichiarati come privati e sono modificabili solo attraverso opportuni metodi



# MODULARITÀ

- Oltre all'astrazione e all'incapsulamento, un principio fondamentale della progettazione è la modularità. I moderni sistemi software consistono tipicamente di diversi componenti diversi che devono interagire correttamente affinché l'intero sistema funzioni correttamente. Mantenere queste interazioni richiede che questi diversi componenti siano ben organizzati.
- Nella progettazione orientata agli oggetti, questa strutturazione del codice è incentrata sul concetto di modularità.
- La modularità si riferisce ad un principio organizzativo per il codice in cui i diversi componenti di un sistema software sono divisi in unità funzionali separate.



# CLASSE

- Una classe consiste di membri.
- I membri che sono variabili o costanti sono membri dati (chiamati anche membri variabili) e i membri che sono funzioni sono chiamati membri funzioni (o anche metodi).
- I membri posso essere di ogni tipo, possono a loro volta essere delle classi, o puntatori o riferimenti a classi.
- I metodi possono agire sui membri variabili e definire il comportamento della classe.



# CLASSE

```
public class EsempioClasse{  
    private int numero;  
    private String testo;  
  
    public EsempioClasse(int numero, String testo){  
        this.numero=numero;  
        this.testo=testo;  
    }  
  
    public void mostraDettagli(){  
        System.out.println("Numero" +numero+ ", testo: "+testo);  
    }  
  
    public static void main(String []args){  
        }  
}
```



# SPECIFICATORI DI ACCESSO

- I membri di una classe sono classificati in tre categorie: **private**, **public** e **protected**, chiamati specificatori di accesso ai membri.
- Per default, tutti i membri di una classe sono privati.
- Se un membro di una classe è privato, non è possibile accedervi al di fuori della classe.
- Un membro pubblico è accessibile al di fuori della classe.
- Per rendere pubblico un membro di una classe, è necessario usare lo specificatore di accesso al membro **public** con i due punti.



# STRINGHE

Java offre una gestione avanzata delle stringhe con la classe `String`.

```
String saluto = "Ciao, mondo!";  
System.out.println(saluto.length()); // Output: 12
```

Le stringhe possono essere concatenate con `+` e confrontate con `.equals()`.

```
String s1 = "Java";  
String s2 = "Java";  
System.out.println(s1.equals(s2)); // Output: true
```

# Costruttori

I costruttori vengono usati per inizializzare oggetti.

```
class Persona {  
    private String nome;  
  
    public Persona(String nome) {  
        this.nome = nome;  
    }  
}
```



# Ereditarietà

Java supporta l'ereditarietà con `extends`

```
class Animale {  
    void faiSuono() {  
        System.out.println("Suono generico");  
    }  
}  
  
class Cane extends Animale {  
    void faiSuono() {  
        System.out.println("Bau Bau");  
    }  
}
```



# Polimorfismo

Il polimorfismo permette di trattare oggetti di classi diverse in modo uniforme.

```
class Forma {  
    void disegna() {  
        System.out.println("Disegnando una forma generica");  
    }  
}  
  
class Cerchio extends Forma {  
    void disegna() {  
        System.out.println("Disegnando un cerchio");  
    }  
}
```



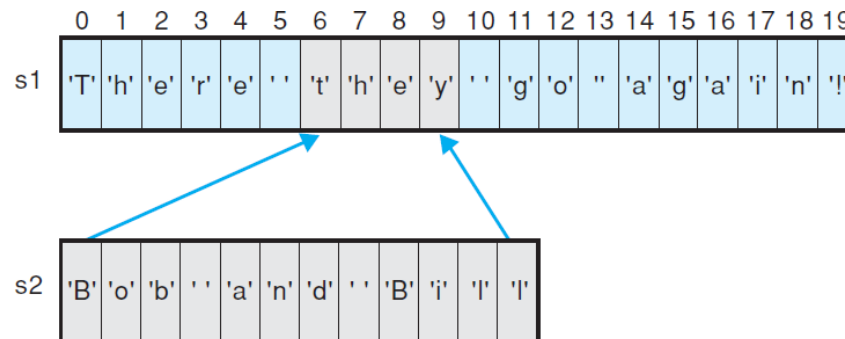
# CONFRONTO, INSERIMENTO, CANCELLAZIONE

- Le stringhe sono confrontate lessicograficamente, carattere per carattere. Un confronto restituisce un valore di tipo bool
- Inserimento: `string s1("Miss Summer");`
- `s1.insert(5, "Ashley "); // Insert at position: 5`
- Qual è l'output?
- Cancellazione
- `string s("The summer-time");`
- `s.erase(4,7); // Start position: 4, Quantity: 7`

# SOSTITUZIONE

- **Example “Bob and Bill”**

- `string s1("There they go again!");`
- `s2("Bob and Bill");`
- `s1.replace(6, 4, s2);`



# RICERCA

- **Example:** `string youth("Bill is so young, so young");`
- `int first = youth.find("young");`
- La variabile `first` ha valore 11.
- Esiste anche il metodo destro `rfind()`
- Il valore restituito è 21:
- **Example:** `int last = youth.rfind("young");`

# REPLACING

- **Example:** `string s1("Here comes Mike!"),`
- `s2("my love?");`
- `s1.replace(11, 4, s2, 0, 7);`
- La stringa diventa `s1 "Here comes my love!"`.

# ACCESSO ALLA STRINGA

- **Esempio:** `s.at(i) = 'X';` is equivalent to `s[i] = 'X';`

# OVERLOADING DI FUNZIONI O METODI

- Overloading vuol dire che funzioni o operatori hanno lo stesso nome, ma argomenti o operandi diversi e il cui effetto dipende dai tipi dei loro operandi o argomenti reali.

## Overloading di funzioni

In Java è possibile definire più funzioni con lo stesso nome purché accettino diversi argomenti in input.

- Facciamo un esempio
- Questo risulta molto utile nel caso di funzioni hanno fini simili ma compiono diverse operazioni perché operano su argomenti di tipo diverso. Al momento della chiamata il compilatore stabilisce quale funzione invocare in base all'argomento. Ad esempio

```
class Calcolatrice {  
    // Somma di due numeri interi  
    public int somma(int a, int b) {  
        return a + b;  
    }  
  
    // Somma di tre numeri interi  
    public int somma(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Somma di due numeri a virgola mobile  
    public double somma(double a, double b)  
    {  
        return a + b;  
    }  
}
```



# ALTRO SEMPLICE ESEMPIO

- Supponiamo di voler definire una classe, `clockType`, per implementare l'ora in un programma.
- Inoltre, supponiamo che l'ora sia rappresentata come un insieme di tre interi: uno per rappresentare le ore, uno per rappresentare i minuti e uno per rappresentare i secondi.
- Vogliamo anche eseguire le seguenti operazioni:
  - 1. Impostare l'ora.
  - 2. Restituire l'ora.
  - 3. Stampare l'ora.
  - 4. Aumenta il tempo di un secondo.
  - 5. Incrementa il tempo di un minuto.
  - 6. Incrementa il tempo di un'ora.
  - 7. Confronta due tempi per l'uguaglianza.



# CLASSE CLOCKTYPE

```
class Orologio {  
    private int ore, minuti, secondi;  
  
    public Orologio(int ore, int minuti, int secondi) {  
        this.ore = (ore >= 0 && ore < 24) ? ore : 0;  
        this.minuti = (minuti >= 0 && minuti < 60) ? minuti : 0;  
        this.secondi = (secondi >= 0 && secondi < 60) ? secondi : 0;  
    }  
  
    public void incrementaSecondi() {  
        secondi++;  
        if (secondi >= 60) {  
            secondi = 0;  
            incrementaMinuti();  
        }  
    }  
  
    private void incrementaMinuti() {  
        minuti++;  
        if (minuti >= 60) {  
            minuti = 0;  
            incrementaOre();  
        }  
    }  
}
```



# USARE GLI OGGETTI

```
private void incrementaOre() {  
    ore = (ore + 1) % 24;  
}  
  
public void mostraTempo() {  
    System.out.printf("%02d:%02d:%02d\n", ore, minuti, secondi);  
}  
}
```



# COSTRUTTORI

- Per garantire che le variabili di istanza di una classe siano inizializzate, si usano costruttori.
- Ci sono due tipi di costruttori: con parametri e senza parametri. Il costruttore senza parametri è chiamato costruttore di default.
- I costruttori hanno le seguenti proprietà:
  - Il nome di un costruttore è uguale al nome della classe.
  - Un costruttore, anche se è una funzione, non ha un tipo, cioè, non è né una funzione che restituisce un valore né una funzione void.
  - Una classe può avere più di un costruttore. Tuttavia, tutti i costruttori di una classe hanno lo stesso nome. Se una classe ha più di un costruttore, i costruttori devono avere diversi elenchi di parametri formali.
  - I costruttori vengono eseguiti automaticamente quando un oggetto di classe viene istanziato. Quale costruttore viene eseguito dipende dai tipi di valori passati all'oggetto quando l'oggetto classe viene dichiarato.



# OGGETTI

- Una variabile di una classe è chiamata oggetto di classe o istanza di classe, o più comunemente **oggetto**
- La sintassi è `className classObjectName;`
- Quando dichiarare un oggetto viene eseguito il costruttore
- ESEMPIO: La dichiarazione **clockType myClock;** dichiara che myClock è un oggetto di tipo clockType. In questo caso, il costruttore predefinito viene eseguito e le variabili di istanza di myClock sono inizializzate a 0.
- La sintassi per dichiarare un oggetto che invoca un costruttore con parametri è:

`className classObjectName(argument1, argument2, ...);`



# ACCESSO AI MEMBRI

- Esempio:

```
clockType myClock;  
clockType yourClock;
```

- Tre istruzioni legali sono:

```
myClock.setTime(5, 2, 30);  
myClock.printTime();  
if (myClock.equalTime(yourClock)) ...
```

- Sono illegali:

```
myClock.hr = 10; //illegal  
myClock.min = yourClock.min; //illegal
```



# PUNTATORE THIS

Dato actObj l'oggetto corrente della classe Class\_id, il puntatore **this** ha la seguente dichiarazione implicita

- `Class_id* const this = &actObj;`



# EREDITARIETÀ

- L'ereditarietà ci permette di creare nuove classi da classi esistenti. Le classi esistenti sono chiamate le **classi base**; la nuova classe che creiamo dalle classi esistenti è chiamata **classe derivata**.
- La classe derivata eredita le proprietà delle classi base. Quindi, piuttosto che creare classi completamente nuove da zero, possiamo trarre vantaggio dall'ereditarietà e ridurre la complessità del programma.
- Ogni classe derivata, a sua volta, diventa una classe base per una futura classe derivata.
- Ereditarietà può essere sia un'eredità singola che un'eredità multipla. In un'eredità singola, la classe derivata deriva da una singola classe base; in un'eredità multipla, la classe derivata è derivata da più di una classe base



# CLASSI DERIVATE

- I membri privati di una classe base sono privati della classe base; quindi, i membri della classe derivata non possono accedervi direttamente. In altre parole, quando scrivete le definizioni delle funzioni membro della classe derivata, non potete accedere direttamente ai membri privati della classe base.
- I membri pubblici di una classe base possono essere ereditati come membri pubblici o come membri privati dalla classe derivata. Cioè, i membri pubblici della classe base possono diventare membri pubblici o privati della classe derivata.
- La classe derivata può includere membri-dati e/o funzioni aggiuntive.
- La classe derivata può ridefinire (overriding) le funzioni pubbliche dei membri della classe base. Cioè, nella classe derivata, si può avere una funzione membro con lo stesso nome, numero e tipo di parametri di una funzione nella classe base. Tuttavia, questa ridefinizione si applica solo agli oggetti della classe derivata, non agli oggetti della classe base.
- Tutte le variabili membro della classe base sono anche variabili membro della classe derivata. Allo stesso modo, le funzioni membro della classe base (a meno che ridefinite) sono anche funzioni membro della classe derivata.
- Normalmente sono memorizzate in appositi file header



# ESEMPIO CLASSE DERIVATA

```
class Rectangle {  
    // Campi privati per lunghezza e larghezza  
    private double length;  
    private double width;  
  
    // Costruttore di default  
    public Rectangle() {  
        this.length = 0;  
        this.width = 0;  
    }  
  
    // Costruttore con parametri  
    public Rectangle(double length, double width) {  
        setDimension(length, width);  
    }  
  
    // Metodo per impostare lunghezza e larghezza  
    public void setDimension(double length, double width) {  
        this.length = (length >= 0) ? length : 0;  
        this.width = (width >= 0) ? width : 0;  
    }  
}
```



# ESEMPIO CLASSE DERIVATA

```
class Box extends Rectangle {  
    // Campo privato per l'altezza  
    private double height;  
  
    // Costruttore di default  
    public Box() {  
        super(); // Invoca il costruttore di default di Rectangle  
        this.height = 0;  
    }  
  
    // Costruttore con parametri  
    public Box(double length, double width, double height) {  
        super(length, width); // Inizializza lunghezza e larghezza  
        setDimension(length, width, height);  
    }  
}
```



# OVERRIDE NELLA CLASSE DERIVATA

```
class Animale {  
    public void faiSuono() {  
        System.out.println("Suono generico");  
    }  
}  
  
// Classe derivata che esegue l'override del metodo faiSuono  
class Cane extends Animale {  
    @Override  
    public void faiSuono() {  
        System.out.println("Bau Bau");  
    }  
}
```



1. Scrivere un programma in Java che
  - Si calcoli la trasposta di una matrice quadrata
  - Si Visualizzi tale matrice
2. Implementare in java gli algoritmi per calcolare la sottosequenza massimale di un vettore, allocato dinamicamente.
3. Scrivere un programma in java che, dato un vettore di interi allocato dinamicamente e riempito con valori casuali compresi tra -100 e 100, trovi il numero delle triple di elementi la cui somma è uguale a zero. Qual è la complessità di tempo della vostra soluzione?
  - Esempio: 30, -40, -20, -10, 40, 0, 10, 5

# SOLUZIONE (PARTE 1)

```
import java.util.Scanner;

public class MatriceTrasposta {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Inserisci la dimensione della matrice quadrata: ");
        int n = scanner.nextInt();

        int[][] matrice = new int[n][n];
        int[][] trasposta = new int[n][n];

        System.out.println("Inserisci gli elementi della matrice:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print("Elemento [" + i + "][" + j + "]: ");
                matrice[i][j] = scanner.nextInt();
            }
        }
    }
}
```

# SOLUZIONE (PARTE 1)

```
// Calcolo della trasposta
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        trasposta[j][i] = matrice[i][j];
    }
}

System.out.println("\nMatrice originale:");
stampaMatrice(matrice);

System.out.println("\nMatrice trasposta:");
stampaMatrice(trasposta);

scanner.close();
}

public static void stampaMatrice(int[][] matrice) {
    for (int[] riga : matrice) {
        for (int elemento : riga) {
            System.out.print(elemento + " ");
        }
        System.out.println();
    }
}
}
```

# SOLUZIONE (PARTE 2)

```
public static int[][] calcolaTrasposta(int[][] matrice) {  
    int n = matrice.length;  
    if (n == 0) return new int[0][0];  
    int[][] trasposta = new int[n][n];  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            trasposta[j][i] = matrice[i][j];  
        }  
    }  
    return trasposta;  
}
```

```
public static void stampaMatrice(int[][] matrice) {  
    if (matrice.length == 0) {  
        System.out.println("Matrice vuota.");  
        return;  
    }  
    for (int[] riga : matrice) {  
        for (int elemento : riga) {  
            System.out.print(elemento + " ");  
        }  
        System.out.println();  
    }  
}
```

```
for(int i = 0; i < n; ++i) {  
    delete[] mat[i];  
}  
delete[] mat;
```

Cancellazione

| Complessità temporale | Complessità spaziale                        |
|-----------------------|---------------------------------------------|
| $O(n^2)$              | $O(n^2)$<br>$O(1)$ nel caso della trasposta |

Eliminare una **riga** è spesso più “naturale” perché le righe sono contigue in memoria (row-major).

Eliminare una **colonna** richiede operazioni su blocchi non contigui meno cache-friendly.

